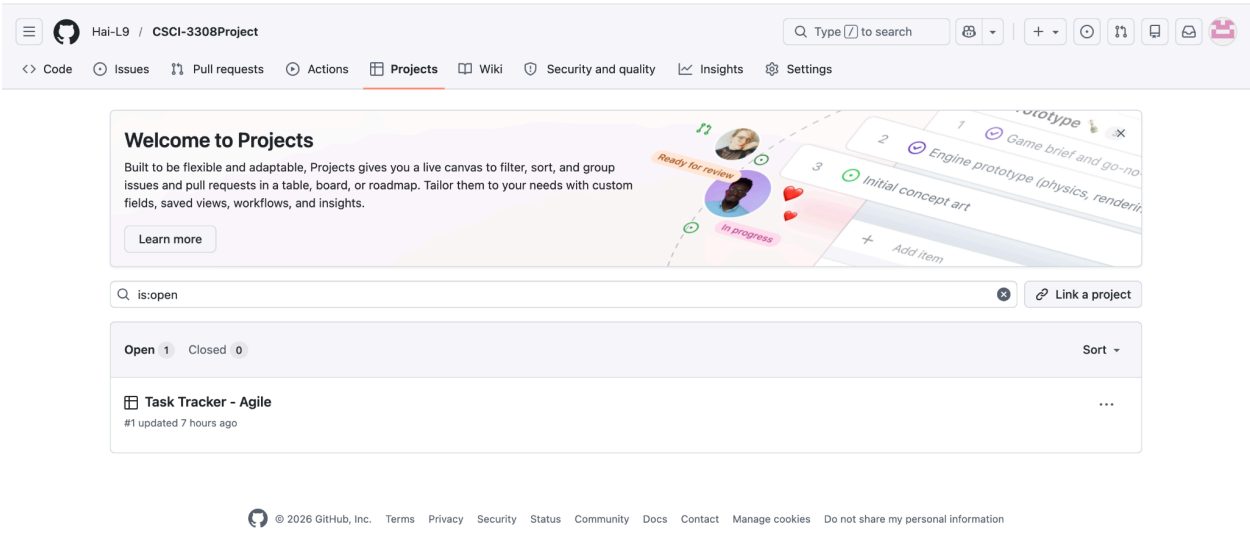
Task Tracker Report

Who: Hudson S, Winston T, Hudson V, Ryken A, Denzel G, Joshua S

Description: Task Tracker is a Dockerized work-order management app for coordinating tasks across real-world worksites. It combines a Kanban task board, role-aware task access, Google Maps location tools, and a manager map dashboard to streamline work order coordination and team communication. The backend uses Express and NodeJS with a PostgreSQL database to manage and store the user and task data. The Docker container ensures that deployment is consistent across devices, both in development and production. To maintain security, the site uses hashed passwords and role based access control. Task visibility and modification rights are limited by the manager or user role.

Github Repo: https://github.com/Hai-L9/CSCI-3308Project/

Project Tracker: https://github.com/users/Hai-L9/projects/1/views/2



Video Demo: https://youtu.be/kP4mkSojifM

Contributions:

Hudson S:

I didn't focus on any one area specifically but my main contributions to the project were the notification system and AI integration. I worked across the application stack to implement these features. I ensured the project / platform could offer both accurate summaries and management help through the AI integration tool as well as enabling proactive communication to employees though the notification system. Throughout the duration of the project I also supported other smaller tasks and cleanup duties throughout the codebase. Additionally, I prioritized code quality and readability throughout my contributions to ensure future scalability and maintenance on this project.

Winston T:

I was more focused on the backend as that is where I am more comfortable. I started with helping on the account creation then moved to Google Maps. The backend uses Express for all API routes and PostgreSQL for the database. A user can assign a physical location or address to a task. The user can change that location and it will maintain the history of worksites. There is a map UI element for dropping pins, and a search bar for finding addresses easier. Near the end, I contributed to the ability to assign a task to a specific user and the associated auto-complete filtering. The middleware is tested in a test suite using Mocha and Chai.

Hudson V:

My main tasks were focused on the frontend and UI/UX of the application. I created the template for the login page, before we moved it to a modal popup. For the main board, I created the role based UI of the Add Task button, including the necessary fields and validations, and making sure that only managers and admins can create new tasks. I also implemented the drag and drop functionality using live cursor tracking, allowing the user to properly keep track of and reorder tasks. I fixed bugs I encountered along the way, such as the permission levels not updating automatically. The final feature I worked on was creating the frontend of the AI assistant that can be injected into any new page we create, making sure that the chats were saved in local storage.

Ryken A:

I first worked on the database and auth side of the project, specifically on maintaining database connection and session persistence. This required working with the people who were building the postgresql database to make sure that we had coherent tables set up and working with the middleware. I then helped with connecting to the database to display our tasks on the kanban board, making sure that the page would properly pull tasks from the board and display them. The last big thing was being able to add and change data in the database. Throughout the project I also made the wireframes, helped on presentation content, and created the architecture diagram.

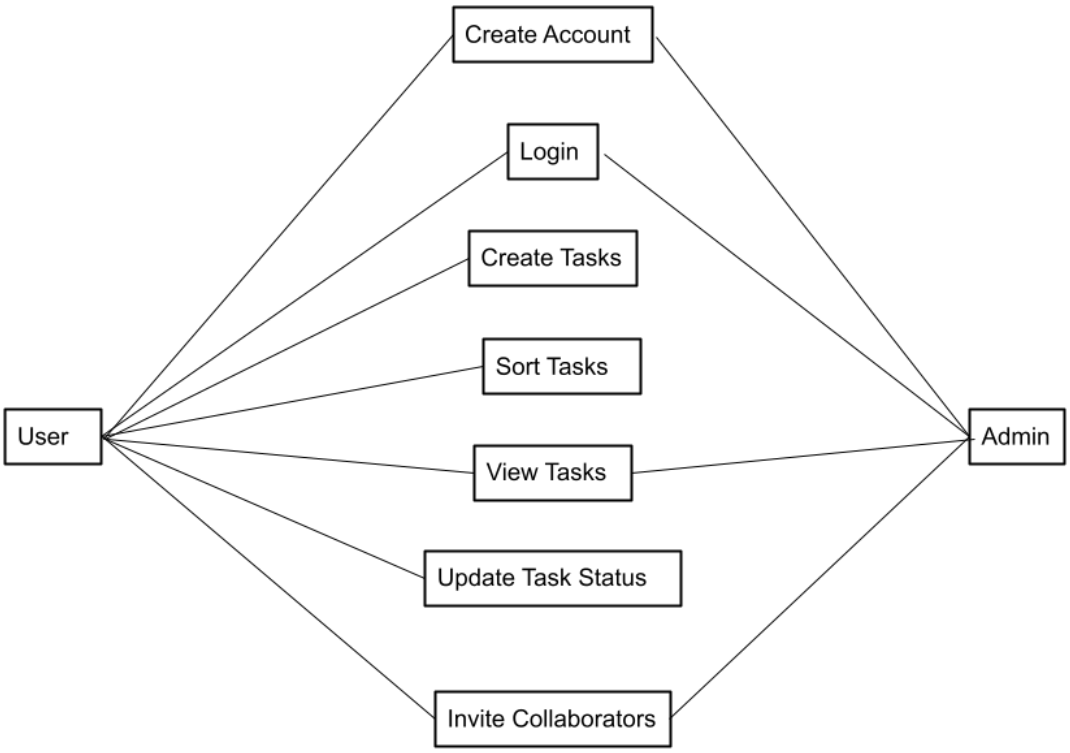
Denzel G:

My contributions to Task Tracker focused primarily on middleware. I helped design and implement the PostgreSQL database schema, creating core tables which included users, tasks, worksites, task_assignments, and session, along with the foreign key relationships and constraints that maintain data consistency. This work was coordinated closely with the team to ensure alignment across the frontend and middleware. I also contributed to the account creation flow by implementing JWT token generation on successful signup, and built out task editing functionality covering title, description, assignee, priority, status, due date, and worksite. My largest contribution was the role-based access control system, which differentiates behavior between admin, manager, and worker accounts, secures API routes with authentication guards, filters task visibility on the backend, and updates the frontend controls based on the logged-in user's role.

Joshua S:

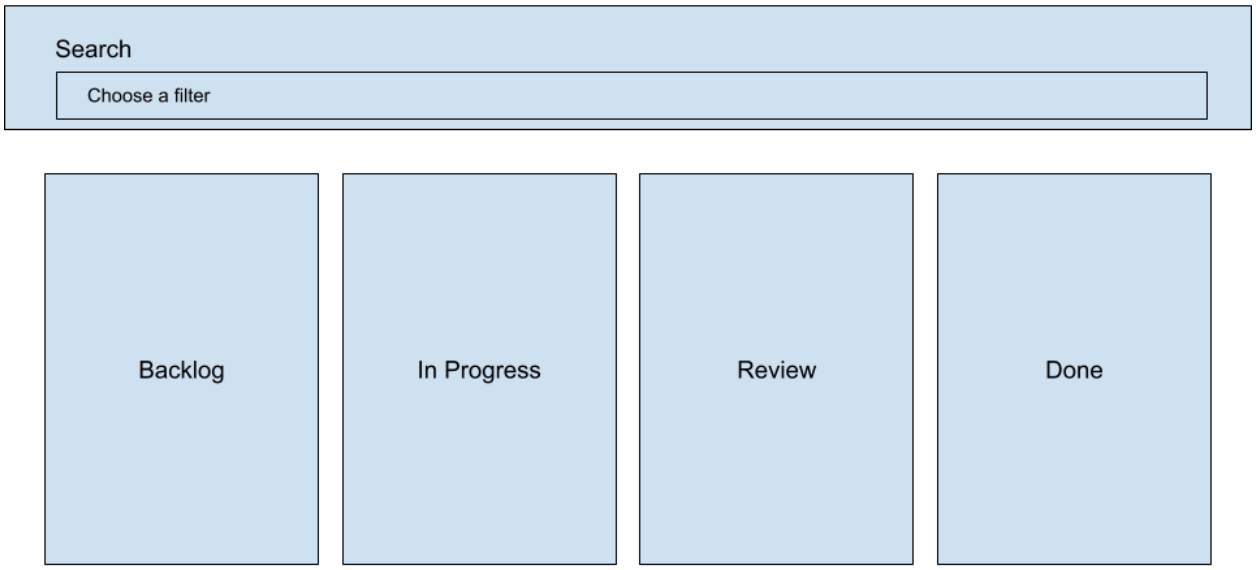
I contributed significantly to the UI and visual design across the project’s pages, along with scripting that enabled search and filter functionality. I also created the map view, which visually renders task locations on an interactive map with informational popups. I also helped with the AI assistant ensuring it could see relevant information when the user requests summaries or help involving tasks. In addition to my development work, I helped lead the team’s project management and organization by writing the initial project breakdown, user stories, sprint goals, MVP goals, and filling out our GitHub project tracker with that information. I also created the team presentation slides and script, as well as the README for our repository.

Use Case Diagram:

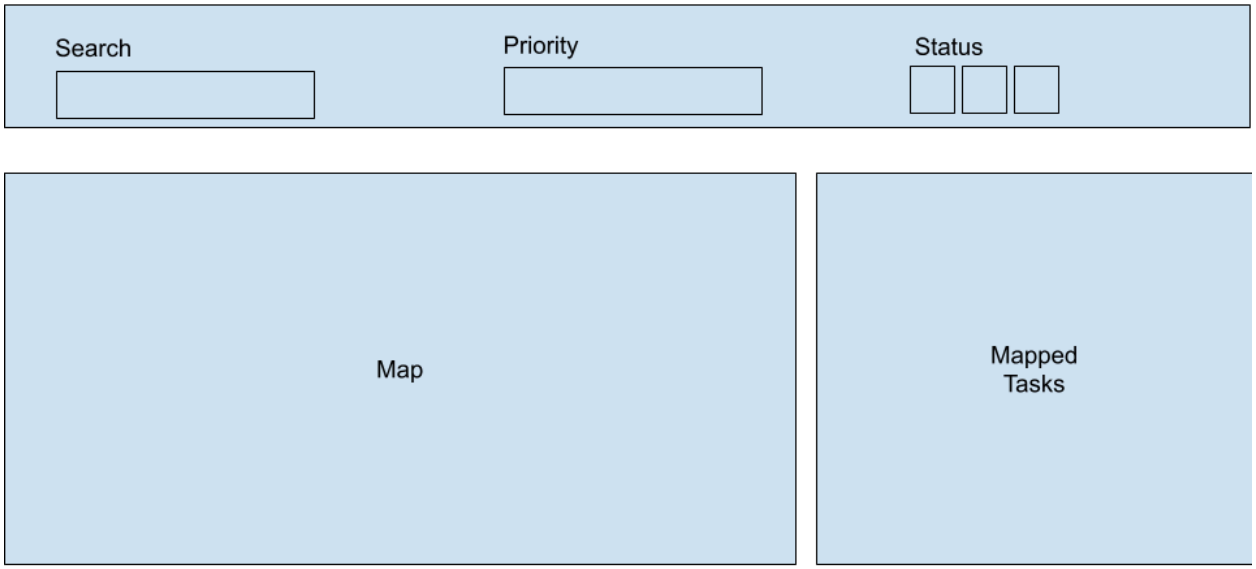


Wireframes:

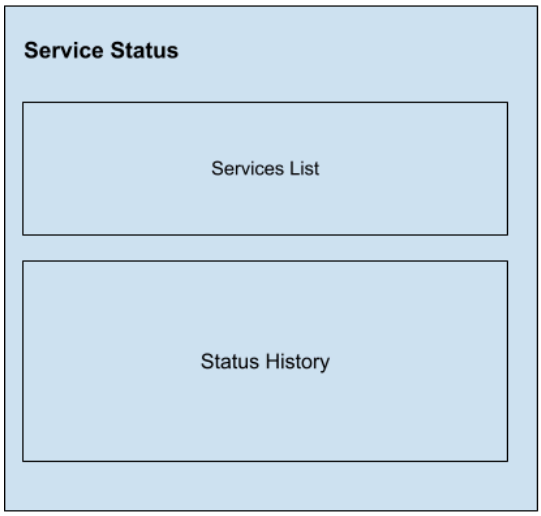
Kanban Board

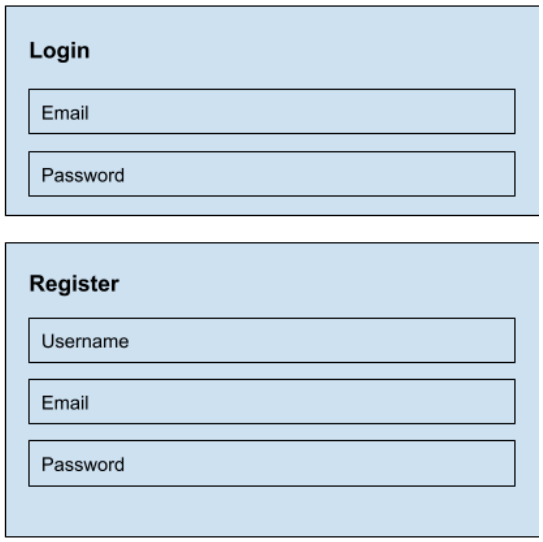


Task Map



Modals


Service Status
Services List
Status History


Login
Email
Password
Register
Username
Email
Password

Test Results:

Automated test suite covering all routes and endpoints in the backend.

1. Authentication and User Profiles

- **Test Suite:** user registration, login, account updates, and session persistence
- **Test Cases:**
    - Valid/Invalid registration and login attempts
    - Duplicate username/email
    - Account updates
    - **Expected Result:** account creation, hashed password storage, and reliable session/token management
- **Observations:** Handles edge cases like duplicate accounts (409) and missing fields (400). Session persistence is tested through a visit counter across multiple requests.

2. Task Management

- **Test Suite:** creation, modification, tracking, and deletion of project tasks
- **Test Cases:**
    - Full CRUD operations
    - Validation of task metadata
    - Task history tracking and Google Maps
- **Expected Result:** Tasks are correctly stored with all metadata, and invalid fields are rejected
- **Observations:** Managers can access and modify everything, and users cannot. Data validation ensures all fields are correct and filled in.

3. Role Based Access

- **Test Suite:** security boundaries between "Manager" and "Worker" roles
- **Test Cases:**
    - Workers attempting to create or delete tasks/worksites
    - Workers attempting to view the full user list or unassigned tasks
    - Unauthenticated users attempting to access protected API endpoints

- **Expected Result:** Unauthorized actions are blocked with 403 (Forbidden) or 401 (Unauthorized) status codes
- **Observations:** Workers are limited to viewing and updating the status of tasks specifically assigned to them.

4. Worksite Management

- **Test Suite:** management of physical worksites
- **Test Cases:**
    - Worksite creation with GPS coordinates
    - API health check, up/down status and successful API key auth
- **Expected Result:** storage of geographic data and reliable/secure usage of environmental variables
- **Observations:** Handles floating-point coordinate data for worksites, and fallback on 403 error for API key related problems

Deployment: Production deployment is available at
https://csci-3308project.onrender.com/